■■ Prompt Injection Firewall

A security layer that intercepts content before it reaches an AI/LLM, detecting and neutralizing prompt-injection attacks while letting legitimate content pass with minimal disruption.

What it is

AI agents, copilots, and chatbots increasingly read external content (web pages, PDFs, emails, API responses) and can call tools. That exposes them to **prompt injection** — hidden instructions that make the AI obey an attacker instead of the user. This firewall sits in front of the model and inspects every input first.

```
incoming content  →  [ FIREWALL ]  →  ALLOW / SANITIZE / BLOCK  →  the AI
```

ALLOW	Looks safe — passes through unchanged.
SANITIZE	Suspicious parts redacted; safe remainder forwarded.
BLOCK	High-risk injection — nothing reaches the AI.

What it detects & reads

9 attack types	Instruction Override, Role Change, Secret Extraction, Tool Abuse, Credential Theft, Context Poisoning, Multi-Step Jailbreaks, Encoded Instructions, Indirect Prompt Injection.
Input sources	Plain text, HTML / web pages, PDF, Email (.eml), Word (.docx), Markdown, source code, API / JSON, Images (OCR, optional).

How it decides — four layers

1. **Rules** — ~30 pattern/heuristic rules over *normalized* text (whitespace collapsed, zero-width characters stripped, full-width folded), so line wraps and typographic tricks don't evade detection.
2. **Decoders** — base64/hex/rot13 payloads are decoded and re-scanned, catching instructions hidden inside encoded blobs.
3. **LLM classifier (optional)** — a semantic second opinion that judges *intent* to catch paraphrases the rules miss; runs only on inconclusive inputs and degrades safely if unavailable.
4. **Session tracker** — spots multi-step jailbreaks built gradually across conversation turns.

Scoring: risk = max(matched severity) + 20×(extra attack types) + up to 15 for repetition. Risk ≥ 70 → BLOCK; ≥ 30 → SANITIZE; otherwise ALLOW (thresholds configurable). Severities are calibrated so an unambiguous attack blocks on its own while ambiguous phrasing needs corroboration.

Measured reliability (honest numbers)

The benchmark counts BLOCKED, FORWARDED, and MISSED separately — a redacted-but-forwarded attack is not scored as a clean win. Two corpora are reported: one the rules were tuned on, and a held-out set they were not.

Corpus	Blocked	Forwarded	Missed	False alarms
corpus.json (tuned on)	54 / 54	0	0	0 / 30
holdout.json (never tuned)	7 / 17	3	7	0 / 13

Read the holdout row. On novel phrasings the rule layer catches ~59% with **0% false positives** (no benign message was ever flagged, including deliberate near-misses). The optional LLM classifier layer is the designed answer for the remaining novel and multilingual attacks.

Setup

Runs fully offline with no API key (rules only). Python 3.10+.

```
pip install -r requirements.txt
uvicorn app.main:app --reload
# open http://127.0.0.1:8000
```

Enable the LLM layer (optional) — create a `.env`:

```
OPENAI_API_KEY=your-key
OPENAI_BASE_URL=https://YOUR-RESOURCE.openai.azure.com/openai/v1
OPENAI_MODEL=your-deployment-name  # Azure: deployment, not model
FIREWALL_LLM_MODE=all
```

Integrating with an existing chatbot / app

As a service (any language) — POST to the firewall before calling your own LLM; block when the verdict is BLOCK, forward `sanitized_text` when it is SANITIZE:

```
r = httpx.post('http://127.0.0.1:8000/api/scan',
    json={'content': user_text, 'source_type': 'text'})
if r.json()['verdict'] == 'BLOCK': refuse()
else: my_llm(user_text)
```

In-process (Python) — import the engine directly:

```
from app.firewall import Firewall, Verdict
result = Firewall().scan(user_message, session_id=sid)
if result.verdict == Verdict.BLOCK: refuse()
else: my_llm(result.sanitized_text or user_message)
```

Guard retrieved content (RAG / agents) — the high-value case: run every web page, PDF, or email through the extractor + firewall *before* it enters the model's context. Indirect injection lives in retrieved content, which is the easiest path to forget.

Scope & known limitations

This is a hackathon prototype. Current gaps, documented honestly: the `firewall_enabled` / `system_prompt` request-body flags are unauthenticated and should be server-side in production; there is no output-side scanning yet (inputs only); email HTML alternatives and attachments, percent/entity encodings, and multi-layer base64 are not fully handled; non-English attacks rely on the LLM layer. The rule layer is strong on known attack families with zero false positives; the LLM layer covers novel and multilingual phrasings.